

```
DATA_PATH = Path("FiresRu.csv")
df = pd.read_csv(DATA_PATH)

print(df.shape)
print(df.columns.tolist())
df.head()
```

Результат выполнения

```
(26681, 9)
['type_name', 'type_id', 'lon', 'lat', 'temperature_c',
'precipitation_mm', 'relative_humidity', 'wind_speed_ms',
'solar_radiation']
```

Таблица 3. Результат выполнения блока 4.1. в виде таблицы

type_name	type_id	lon	lat	temperature_c	precipitation_mm	relative_humidity	wind_speed_ms	solar_radiation
Natural fire	4	131.59	47.87	-9.44	0.01	72.98	1.14	4.59
Natural fire	4	131.59	47.88	-9.44	0.01	72.98	1.14	4.59
Forest fire	3	131.99	48.50	-10.71	0.05	81.31	0.80	4.24
Natural fire	4	131.90	43.63	-7.77	0.00	68.41	0.97	4.66
Natural fire	4	131.57	47.86	-9.44	0.01	72.98	1.14	4.59

Загрузка прошла без явного задания `sep` и `encoding`: файл использует стандартный разделитель «,» и читается как таблица из 26 681 строк и 9 столбцов. По перечню полей сразу видно отсутствие временной метки — значит, временной анализ на этом наборе не является корректной задачей.

5. Первичная диагностика данных

Первичный анализ до построения графиков и вычисления сложных метрик ставит своей целью выяснить какой размер у входной таблицы данных, какие типы данных в атрибутах распознаны, есть ли пропуски и дубликаты в данных, какие диапазоны значений наблюдаются.

Метод `head()` показывает примеры строк, `info()` — структуру и память, `describe()` — основные статистики, `isna()` и `duplicated()` — простейшие индикаторы качества.

Практический блок 5.1. Структура и память

Вместо непосредственного вывода `info()` в интерфейс Notebook его результат будет сначала перехвачен в строковый буфер, а затем напечатан. Это удобно для

воспроизводимого отчёта и одновременно позволяет изучить структуру таблицы и фактическое потребление памяти. По результату необходимо обратить внимание не только на отсутствие пропусков, но и на тип object у категориального столбца `type_name`, поскольку именно он далее станет кандидатом на оптимизацию.

Модуль `io` входит в стандартную библиотеку Python. Объект `io.StringIO()` создаёт текстовый поток в оперативной памяти и может использоваться как временный «файл».

Метод `df.info()` выводит сводку о `DataFrame`: тип индекса, число строк, имена столбцов, количество непустых значений и `dtypes`. Параметр `buf` перенаправляет этот вывод в созданный поток, а `memory_usage="deep"` просит Pandas глубже учитывать память, занятую объектными значениями. Метод `getvalue()` извлекает накопленный текст из `StringIO`.

```
import io

buf = io.StringIO()
df.info(buf=buf, memory_usage="deep")
print(buf.getvalue())
```

Результат выполнения

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 26681 entries, 0 to 26680
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   type_name              26681 non-null  object
1   type_id                26681 non-null  int64
2   lon                    26681 non-null  float64
3   lat                    26681 non-null  float64
4   temperature_c          26681 non-null  float64
5   precipitation_mm        26681 non-null  float64
6   relative_humidity       26681 non-null  float64
7   wind_speed_ms           26681 non-null  float64
8   solar_radiation         26681 non-null  float64
dtypes: float64(7), int64(1), object(1)
memory usage: 3.2 MB
```

Тип `object` у `type_name` означает строковые Python-объекты. Для столбца с пятью повторяющимися категориями это функционально корректно, но избыточно по памяти. Ниже он будет преобразован в `category`.

Практический блок 5.2. Пропуски, дубликаты и уникальные значения

Следующий код выполняет три независимые проверки качества данных: полноту, наличие полных дублей и кардинальность признаков. Сначала будет подсчитано число пропусков по каждому полю, затем — число полностью совпадающих строк, после чего для всех столбцов определяется количество уникальных

значений. Кардинальность особенно важна для распознавания категориальных признаков и для обнаружения полей, которые внешне имеют числовой dtype, но фактически являются кодами категорий.

Метод `isna()` строит DataFrame логических значений: `True` соответствует пропуску, `False` — заполненному элементу.

Последующий `sum()` суммирует `True` как единицы и тем самым даёт число пропусков по каждому столбцу.

`uplicated()` возвращает булеву маску полных повторов строк; её `sum()` считает количество таких повторов.

Метод `nunique()` вычисляет число различных значений отдельно для каждого столбца.

```
print("Пропуски по столбцам:")
print(df.isna().sum())
print("Полных дубликатов:", df.duplicated().sum())
print("Число уникальных значений:")
print(df.nunique())
```

Результат выполнения

```
Пропуски по столбцам:
type_name      0
type_id        0
lon            0
lat            0
temperature_c  0
precipitation_mm 0
relative_humidity 0
wind_speed_ms  0
solar_radiation 0
Полных дубликатов: 0
Число уникальных значений:
type_name      5
type_id        5
lon           25708
lat           23547
temperature_c  2581
precipitation_mm 571
relative_humidity 4028
wind_speed_ms  532
solar_radiation 731
```

В данных нет пропущенных значений и нет полностью совпадающих строк. Это не отменяет семантической проверки. Например, `type_id` необходимо интерпретировать как код категории, а не как величину, для которой осмысленно среднее арифметическое.

Практический блок 5.3. Проверка соответствия кода и категории

Задача этого блока — проверить семантическую согласованность двух представлений одного и того же признака числового `type_id` и текстового `type_name`. Из исходной таблицы будут выделены только эти поля, повторяющиеся пары удалены, а уникальные соответствия отсортированы по коду. Если одному коду соответствовало бы несколько названий или одному названию — несколько кодов, это стало бы заметно в полученной таблице и потребовало бы дополнительной проверки качества данных.

Выражение `df[["type_id", "type_name"]]` выбирает из `DataFrame` два столбца и возвращает новый `DataFrame`.

Метод `drop_duplicates()` удаляет повторяющиеся пары значений, а `sort_values("type_id")` упорядочивает оставшиеся строки по числовому коду.

Круглые скобки позволяют записать цепочку методов на нескольких строках без символа продолжения.

Метод `to_string(index=False)` формирует текстовое представление таблицы без вывода индекса.

```
type_map = (
    df[["type_id", "type_name"]]
      .drop_duplicates()
      .sort_values("type_id")
)
print(type_map.to_string(index=False))
```

Таблица 4. Результат выполнения блока 5.3 в виде таблицы

type_id	type_name
1	Uncontrolled burn
2	Peat fire
3	Forest fire
4	Natural fire
5	Controlled burn

Набор содержит взаимно однозначное соответствие пяти кодов пяти названиям событий, среди которых 1 — Uncontrolled burn, 2 — Peat fire, 3 — Forest fire, 4 — Natural fire, 5 — Controlled burn. Следовательно, `type_id` можно хранить компактным целым типом, но в статистическом анализе рассматривать как категориальный признак.

Практический блок 5.4. Описательная статистика

Следующий код формирует компактную статистическую сводку для координат и метеорологических полей. По ней нужно оценивать центральные значения, разброс и границы диапазона, а также искать признаки, у которых медиана заметно отличается от среднего или максимум сильно удалён от верхнего квартиля.

Список `cols` задаёт явный набор количественных признаков, которые должны участвовать в статистическом описании. Выражение `df[cols]` выбирает только эти столбцы.

Метод `describe()` для числовых данных рассчитывает `count`, `mean`, `std`, `minimum`, квартиль 25%, медиану 50%, квартиль 75% и `maximum`. Метод `round(2)` округляет результат до двух знаков, а свойство `.T` транспонирует таблицу, превращая признаки в строки — такой формат удобнее для чтения широкого набора показателей.

В статистику намеренно не включается `type_id`, хотя технически он имеет целочисленный `dtype`. Это код категории, и среднее значение для него не имеет содержательного смысла.

```
cols = [
    "lon", "lat", "temperature_c", "precipitation_mm",
    "relative_humidity", "wind_speed_ms", "solar_radiation"
]
print(df[cols].describe().round(2).T)
```

Таблица 5. Результат выполнения практического блока 5.4 в виде таблицы

Признак	count	mean	std	min	25%	50%	75%	max
<i>lon</i>	26681.00	106.35	27.31	20.37	82.41	96.56	133.90	175.87
<i>lat</i>	26681.00	57.53	4.30	41.71	53.95	58.97	60.64	70.33
<i>temperature_c</i>	26681.00	17.89	5.68	-34.88	15.70	19.08	21.39	31.94
<i>precipitation_mm</i>	26681.00	0.46	1.46	0.00	0.00	0.04	0.30	25.85
<i>relative_humidity</i>	26681.00	69.44	11.43	24.70	62.93	71.35	77.91	99.83
<i>wind_speed_ms</i>	26681.00	0.63	0.93	0.00	0.11	0.25	0.66	7.42
<i>solar_radiation</i>	26681.00	5.57	1.50	0.42	4.55	5.68	6.71	8.97